

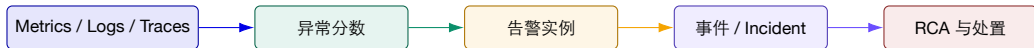
面向 AI 原生系统的智能运维

第 6 讲 | 基于机器学习的故障检测与根因分析 (1/3)

时序异常检测、模型选择、评测陷阱与告警收敛降噪

陈鹏飞 | 中山大学

从第 5 讲继续：从观测数据判断何时需要告警



第 3-5 讲 建立多层可观测数据与调用链。

第 6 讲 判断“何时偏离正常”，并控制告警噪声。

第 7-8 讲 把事件与多源证据推进到根因分析。

检测算例：连续越阈规则用延迟换掉了哪些误报？

按分钟采样。基线均值 100、标准差 10；采用单侧规则 $z>3$ ，连续两点满足才报警。

时刻	观测值	z 分数	告警状态
t1	132	3.2	候选，尚不报警
t2	101	0.1	清除候选
t3	135	3.5	新候选，真实故障从此开始
t4	140	4.0	触发告警

$$z_t = (x_t - 100)/10, \quad \text{检测延迟} = t4 - t3 = 1 \text{ min}$$

推导与判断 规则过滤了 t1 的孤立尖峰，但代价是至少增加一个采样间隔的确认延迟；若故障短于两点，也可能被整段漏掉。

算例使用假设数据。

开场事故：30 万条告警中，值班人员真正需要看到几条？

监控系统看到

- ▶ 2000 个 Pod 同时出现延迟与重试告警；
- ▶ CPU、队列、数据库连接、下游超时接连越线；
- ▶ 每个实例每分钟重复通知；
- ▶ 一次发布恰好发生在告警风暴前。

值班人员需要

- ▶ 一个事件，而不是几万条通知；
- ▶ 影响范围、开始时间和最早异常对象；
- ▶ 哪些信号只是传播结果；
- ▶ 下一步可执行的验证动作。

核心矛盾 检测器负责发现偏离，告警治理负责减少无效告警处理；两者的优化目标并不相同。

先把任务写成四个可证伪问题

Q1 | 何为正常？ 全局范围、同一对象历史、同类对象，还是业务日历下的条件正常？

Q2 | 如何判异常？ 阈值、残差、孤立程度、重构误差或跨对象差异？

Q3 | 如何证明有效？ 点级 F1、事件覆盖、提前量、误报负担还是后续 RCA 收益？

Q4 | 如何进入运维？ 如何去重、聚合、抑制、路由并形成可行动事件？

数据约定

检测要求

告警要求

评测规则

本讲路线图

1. 问题建模：异常、故障、告警、标签和阈值；
2. 统计与预测方法：稳健基线、季节性、EWMA/CUSUM、Prophet；
3. 机器学习与深度方法：Isolation Forest、AE/VAE、Transformer；
4. 研究案例：Minder、ShareAD、MOTSAD 的生产化选择；
5. 评测与上线：区间指标、延迟、漂移、Shadow/Canary；
6. 告警收敛：Alertmanager 机制与 AlertGuardian 生命周期治理。

观测序列 → 异常分数 → 告警状态 → 事件对象

本讲诊断要求：任何“异常”结论都必须给出上下文

问题	至少需要说明	不完整的说法
对象	服务/实例/指标/维度/租户	“CPU 异常”
时间	基线窗、检测窗、持续时间、时区	“刚刚突增”
正常模型	阈值、同类对象、预测或重构依据	“模型认为异常”
异常定义	点、区间、上下文、群体或变化点	“分数超过 0.8”
业务影响	SLO、用户、任务或资源后果	“告警很多”
验证动作	对照、回放、变更关联、恢复观察	“建议关注”

课堂规则 如果不能回答“相对谁、持续多久、影响什么、如何验证”，它只是可疑信号，不是可行动事件。

一、先定义异常检测问题

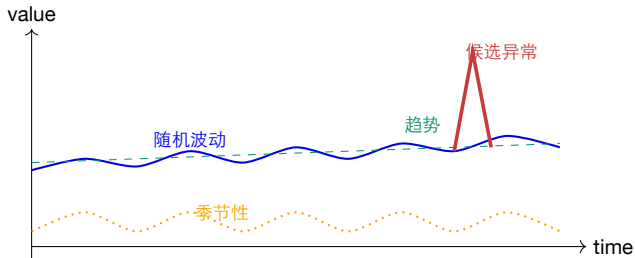
异常不是一个标签，而是“对象、时间、上下文、代价”共同定义的决策。

异常、故障、告警和事件不是同一个概念



- ▶ 正常促销流量可以是统计异常，但不是故障；
- ▶ 静默数据损坏可能是故障，却没有明显指标异常；
- ▶ 同一个故障会触发几千条告警；
- ▶ 同一条告警规则在不同业务时段可能代表不同事件。

一条运维时序至少包含四种结构



若把趋势和季节性直接当噪声，检测器会在每天相同时间稳定误报。

先拆结构，再检测

- ▶ Level: 长期水平;
- ▶ Trend: 缓慢变化;
- ▶ Seasonality: 周期模式;
- ▶ Residual: 尚未解释部分。

异常的四形态：点、上下文、集合与群体

点异常 单点与总体分布显著不同，例如瞬时延迟尖峰。

上下文异常 数值本身常见，但在当前时间/负载/对象上不合理。

集合异常 单点都不极端，但一段序列的形态不正常。

群体异常 同类对象应相似，只有一个实例长期偏离同伴。

Spike

Level Shift

Trend Change

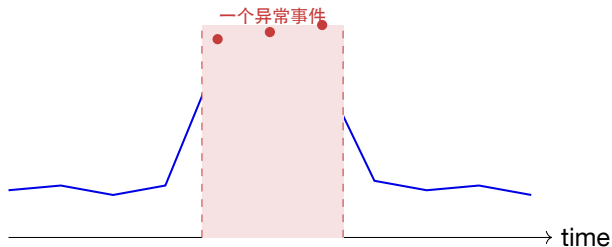
Seasonal Violation

Peer Deviation

运维中的异常形态决定检测方法

形态	例子	优先方法	容易误判
瞬时尖峰 水平迁移 周期破坏 缓慢劣化 多维组合	P99 延迟突然升高 发布后错误率永久抬高 夜间流量模式消失 光功率/磁盘延迟逐周恶化 CPU 略高 + 丢包略高 + 队列略高	稳健 Z-score、窗口分位数 CUSUM、变化点、双窗口 STL/Prophet 残差 趋势模型、斜率与预测区间 Isolation Forest、AE/VAE	抓取抖动、采样错位 业务规模自然增长 节假日、维护窗口 容量扩展、指标口径变化 特征尺度和相关性漂移
同伴偏离	训练集群一台机器吞吐低	Peer similarity + continuity	异构硬件与任务分片

标签单位必须先确定：标一个点，还是标一次事件？



- ▶ 点标签适合训练分数；
- ▶ 区间标签适合事件覆盖；
- ▶ 根因时间早于症状峰值；
- ▶ 人工工单时间通常晚于真实开始；
- ▶ “一次事件算一个 TP” 更接近值班体验。

标签来自告警、工单、注入、专家或恢复动作；来源不同，偏差也不同。

数据质量与归一化：模型之前先处理测量系统

必须检查

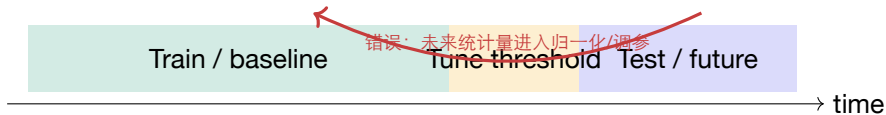
- ▶ 时间戳单调、时区和采样周期；
- ▶ 缺失、重复、延迟到达与 Counter reset；
- ▶ 单位、版本、标签和对象生命周期；
- ▶ 训练窗是否混入已知故障；
- ▶ 插值是否制造“正常曲线”。

归一化选择

- ▶ Standard：均值/方差稳定时；
- ▶ Robust：中位数/MAD 抗离群；
- ▶ Min-Max：边界稳定、无极端漂移时；
- ▶ 按对象、按任务或按同类群体分别建模；
- ▶ 归一化参数必须只从训练基线计算。

比较检测模型之前，先核对采样频率、缺失值、标签和时间窗口是否一致。

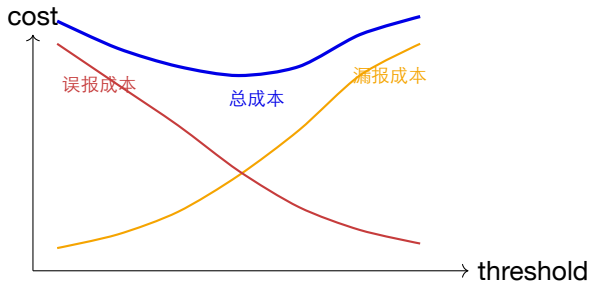
最隐蔽的错误：时间泄漏让离线结果虚高



- ▶ 随机切分破坏时间因果，未来模式泄漏到训练；
- ▶ 用测试集最大/最小值归一化，等于提前知道未来范围；
- ▶ 在测试标签上选择阈值，报告的是“事后最优”；
- ▶ 相邻窗口高度重叠时，训练与测试几乎复制彼此。

正确做法 按时间滚动切分；阈值只在验证窗确定；保留跨业务、跨月份和跨版本外推测试。

阈值不是模型细节，而是成本决策



- ▶ 误报消耗值班注意力；
- ▶ 漏报消耗可用性与业务损失；
- ▶ 不同严重度使用不同阈值；
- ▶ 低流量与高风险对象代价不同；
- ▶ 阈值需要滞回和持续性条件。

同一个异常分数模型，可以因为阈值和持续时间不同而成为完全不同的告警产品。

生产检测链路：分数只是中间产物



模型前 对象、单位、时间、缺失和基线必须可信。

模型后 平滑、滞回、持续性、严重度和业务上下文。

事件后 去重、归并、路由、反馈与真值回填。

二、统计与预测方法

简单方法的优势不是“低级”，而是可解释、可校准、可在小数据下稳定运行。

静态阈值为什么仍然必要，又为什么不够？

它擅长

- ▶ 物理/协议硬边界：温度、容量、错误计数；
- ▶ 明确 SLO：错误率、可用性、P99；
- ▶ 可解释、低成本、易审计；
- ▶ 作为模型故障时的保底规则。

它失败于

- ▶ 季节性与工作日/节假日；
- ▶ 多租户、异构节点与不同负载；
- ▶ 缓慢漂移和组合异常；
- ▶ 阈值附近频繁抖动；
- ▶ 大量规则长期无人维护。

工程结论 硬阈值适合安全边界；自适应基线适合行为偏离；二者通常应并存。

Z-score: 用标准差衡量“离均值多远”

$$z_t = \frac{x_t - \mu}{\sigma}, \quad |z_t| > k \Rightarrow \text{candidate anomaly}$$

适用条件

- ▶ 基线近似平稳;
- ▶ 分布不太重尾;
- ▶ 均值和方差有代表性;
- ▶ 单变量、解释优先。

常见失败

- ▶ 异常本身拉高均值与方差;
- ▶ 周期峰值被稳定误报;
- ▶ 样本量小导致方差不稳定;
- ▶ 不同对象混在一起形成假分布。

Z-score 是“标准化距离”，不是异常概率； $k = 3$ 也不是所有系统的通用真理。

稳健 Z-score：用 Median 与 MAD 抵抗离群污染

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|), \quad z_t^{\text{robust}} = 0.6745 \frac{x_t - \tilde{x}}{\text{MAD}}$$

优势 少量尖峰不容易改变中心和尺度，
适合延迟、队列和吞吐等重尾指标。

边界 MAD 可能为 0；季节性仍未解决；
窗口过短会跟随异常一起移动。

先按对象/时段分组

再计算稳健基线

最后加持续性与滞回

滚动基线：检测“相对最近历史”的偏离

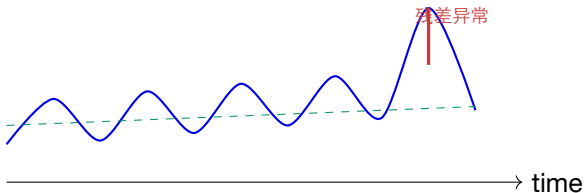
$$\mu_t = \text{mean}(x_{t-w:t-1}), \quad z_t = \frac{x_t - \mu_t}{\sigma_t + \epsilon}$$

- ▶ 窗口太短：基线追随异常，异常很快“变正常”；
- ▶ 窗口太长：对容量增长和版本变化反应迟缓；
- ▶ 窗口包含当前点：会稀释异常分数；
- ▶ 滚动计算应区分冷启动、缺失和低流量；
- ▶ 实践中常用短窗描述当前、长窗描述历史，两者比较。

双窗口思路 短窗显著偏离长窗且持续一段时间，比“瞬时超过固定阈值”更抗噪。

先去除季节性，再在残差上检测

$$x_t = T_t + S_t + R_t, \quad \text{detect on } R_t$$



- ▶ STL: 趋势 + 周期 + 残差;
- ▶ 多周期需日/周/业务日历;
- ▶ 节假日不是普通周末;
- ▶ 周期变化本身也可能是异常。

不要把模型未能解释的部分自动称为故障；残差中仍包含测量噪声和未建模上下文。

EWMA：让近期样本拥有更高权重

$$S_t = \alpha X_t + (1 - \alpha)S_{t-1}, \quad 0 < \alpha \leq 1$$

α 较大 响应快，适合突变；但更容易跟随噪声。

α 较小 曲线稳，适合慢趋势；但检测延迟更长。

- ▶ 可对均值、方差、异常分数或告警状态做 EWMA；
- ▶ 控制图将平滑统计量与上下控制限结合；
- ▶ 适合在线、常数内存场景；
- ▶ 变更后要决定重置基线还是继续继承。

CUSUM: 小偏差持续累积, 最终暴露变化点

$$S_t^+ = \max(0, S_{t-1}^+ + x_t - \mu - k), \quad S_t^- = \max(0, S_{t-1}^- + \mu - x_t - k)$$

- ▶ 单点不明显, 但同方向小偏差会累积;
- ▶ k 表示允许的自然漂移, h 表示触发阈值;
- ▶ 适合检测配置变更后的均值迁移、吞吐缓慢下降;
- ▶ 触发后通常要重置统计量, 并建立新的基线版本;
- ▶ 若存在周期性, 先在残差上使用 CUSUM。

与尖峰检测互补 Z-score 问“这一点有多远”, CUSUM 问“偏差是否持续朝同一方向积累”。

预测残差法：预测正常范围，而不是直接预测异常标签

$$\hat{x}_t = f(x_{<t}, c_t), \quad e_t = x_t - \hat{x}_t$$

- ▶ 模型学习趋势、周期、节假日和外生变量；
- ▶ 实际值落入预测区间时产生异常候选；
- ▶ 预测误差需要按时段/对象校准，不能假设恒定方差；
- ▶ 预测模型失败与系统异常需要区分；
- ▶ 置信区间过窄会误报，过宽会漏报。

Forecast + Residual + Calibrated Interval → Candidate Event

Prophet: 把趋势、季节性、节假日拆成可解释分量

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

适合

- ▶ 业务流量、工单量、日/周周期;
- ▶ 存在节假日和趋势变化;
- ▶ 需要可解释分量;
- ▶ 以预测区间构造残差告警。

不自动解决

- ▶ 高频秒级抖动;
- ▶ 强多变量相关;
- ▶ 突发结构变化;
- ▶ 异常标签与告警代价;
- ▶ 模型更新和阈值漂移。

Prophet 官方提醒：离群点会污染趋势变化与不确定区间

- ▶ 历史离群点可能被模型解释成趋势变化；
- ▶ 未来不确定区间因此被异常放大；
- ▶ 一组极端点还可能扭曲季节性；
- ▶ 官方建议对已确认坏数据置为缺失，Prophet 可处理缺失时间点；
- ▶ 但生产检测不能先删除“未知异常”再评估，否则形成标签泄漏。

正确区分 数据损坏可剔除；真实业务异常必须保留并进入事件标签。二者需要独立的数据质量标记。

多窗口确认：兼顾快速发现与稳定判断

快速窗口 1-5 分钟发现突发；阈值高，要求强信号。

慢速窗口 30 分钟-数小时发现持续劣化；阈值低，要求长期偏离。

组合条件 短窗与长窗同时满足，避免单点噪声和短暂恢复。

低流量保护 样本量不足时不直接计算比例，转为绝对计数或延迟决策。

多窗口思想可用于 SLO burn rate，也可用于异常分数、变化点和群体偏离。

方法选择：从数据结构与运维代价出发

方法	最适合	优势	关键风险
静态/动态阈值 稳健统计 Prophet/预测残差 Isolation Forest AE/VAE Transformer Peer similarity	有明确业务边界 单变量、重尾、少标签 趋势与周期显著 多维表格窗口特征 多变量非线性模式 长依赖、复杂关联 同构并行对象	简单、可解释、低延迟 小数据稳定、易校准 分量可解释、支持日历 无监督、训练快 表征能力强 捕获时间/变量关系 不需固定全局正常范围	规则数量和漂移维护 季节性与上下文不足 高频、多变量和结构突变 时间结构依赖特征工程 重构异常、阈值和漂移 训练成本、数据与评测偏差 异构节点和共同故障

教学案例：数据库延迟每天 10:00 上升，今天是否异常？

1. 固定阈值：每天 10:00 都告警；
2. 滚动均值：如果窗口跨越日周期，基线仍错误；
3. 同时段历史：与过去 4 周周五 10:00 比较；
4. Prophet/STL：剥离周周期与促销日历，在残差上检测；
5. 多变量确认：连接池、队列、错误率和吞吐是否共同变化；
6. 告警决策：只有残差持续且影响 SLO 才 Page，否则进入趋势观察。

课堂思考 若今天是版本发布日，应该把“发布”作为外生变量、抑制条件，还是根因候选？

三、机器学习与深度时序检测

更强的模型能表达复杂正常模式，但并不会自动解决阈值、标签和上线成本。

把时序变成机器学习特征：窗口决定模型能看到什么

窗口统计

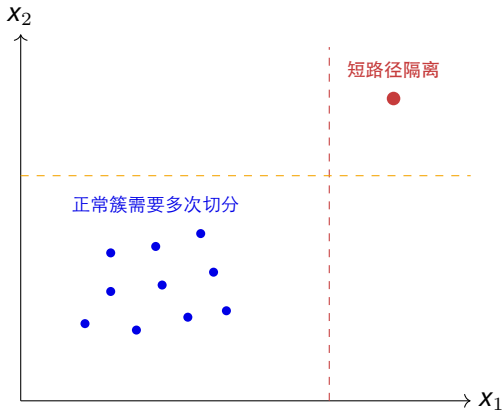
- ▶ mean/std/min/max/quantile;
- ▶ slope、差分、变化率;
- ▶ 峰度、偏度、缺失率;
- ▶ 周期位置、业务日历;
- ▶ 与同伴/上游/下游的差值。

窗口选择

- ▶ 太短看不到慢变化;
- ▶ 太长稀释短异常;
- ▶ 重叠窗口导致样本相关;
- ▶ 多尺度窗口常比单尺度稳;
- ▶ 特征计算时间也属于检测延迟。

Isolation Forest 等“非时序模型”并非不能做时序检测，关键在于窗口与上下文特征。

Isolation Forest: 异常更容易被随机切分快速隔离



- ▶ 随机选择特征;
- ▶ 随机选择切分值;
- ▶ 异常点平均路径更短;
- ▶ 多棵树平均形成分数;
- ▶ 不需要异常标签。

Isolation Forest 的几个生产参数

参数	当前 sklearn 默认/含义	运维解释
n_estimators	100	更多树通常更稳，但训练/内存增加
max_samples	auto=min(256,n)	小子样本体现算法效率，也可能漏掉稀有正常模式
contamination	auto	用于确定分数阈值；真实异常率未知时仍需外部校准
max_features	1.0	特征子采样改变速度和稳定性
random_state	None	生产回放和对比应固定随机种子

注意 模型输出的是相对孤立程度；“contamination=1%”不是系统每天必然有 1% 故障。

Isolation Forest 看不到时间顺序，除非你把顺序编码进去

同一组数值 序列 A: 1,1,1,5,1; 序列 B: 1,5,1,1,1。表格分布相同。

不同运维含义 A 可能是瞬时峰值; B 可能是故障后快速恢复。时间位置影响告警与诊断。

- ▶ 加入 lag、差分、斜率、持续时间与周期位置;
- ▶ 对窗口而非单点建模;
- ▶ 训练集应覆盖不同正常工况;
- ▶ 分数漂移需独立监控;
- ▶ 若异常是“形态”，考虑序列模型或 Matrix Profile。

预测模型：用未来预测误差发现异常



- ▶ 优点：自然表达时间依赖，可给出提前预测；
- ▶ 风险：复杂但可预测的异常可能被正确预测，从而分数低；
- ▶ 模型更新后残差分布改变，阈值必须重新校准；
- ▶ 多步预测要区分每个 horizon 的误差分布。

Autoencoder: 重构“正常流形”，以重构误差计分

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z), \quad s(x) = \|x - \hat{x}\|$$

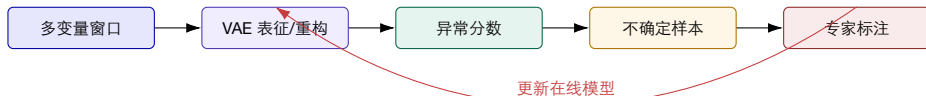
为什么有效

- ▶ 正常样本占多数；
- ▶ 低维空间保留共同结构；
- ▶ 异常模式难以准确重构；
- ▶ 可输出维度级误差贡献。

为什么会失败

- ▶ 容量过大把异常也重构好；
- ▶ 训练数据被异常污染；
- ▶ 新正常工况产生高误差；
- ▶ 误差阈值跨对象不可直接复用。

VAE 与半监督主动学习：让少量专家反馈用在不确定样本上



- ▶ SLA-VAE 将半监督 VAE 与主动学习结合；
- ▶ 目标是用少量不确定样本反馈适配具体微服务/云服务器；
- ▶ 论文在腾讯两类游戏业务云服务器数据上评估；
- ▶ 教学启示：专家预算应投向“能改变决策边界”的样本。

深度时序模型主要在学习三类关系

时间关系 当前是否违背短期、长期或周期模式？

变量关系 CPU、延迟、队列、网络之间的联合模式是否破坏？

上下文关系 对象、任务、负载、拓扑和变更条件是否一致？

- ▶ RNN/LSTM：顺序与长期依赖；
- ▶ TCN：并行卷积和多尺度感受野；
- ▶ Graph model：变量/服务/对象关系；
- ▶ Transformer：长序列与注意力关系；
- ▶ 混合模型：重构 + 预测 + 对比/关联分数。

TranAD: 两阶段重构, 让模型聚焦难以重构的位置

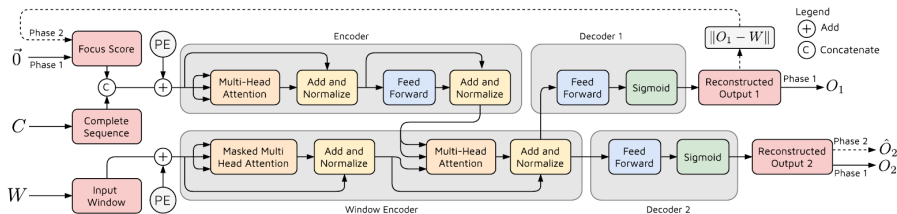
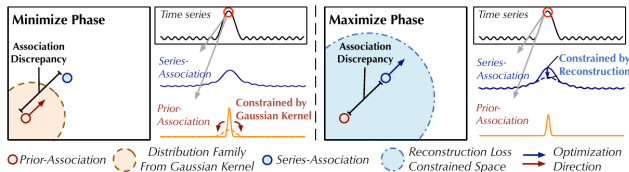


Figure 1: The TranAD Model.

- ▶ 完整序列提供广域上下文，窗口编码器聚焦局部；
- ▶ Phase 1 产生重构误差，Phase 2 将其作为 Focus Score；
- ▶ 目标是兼顾时间关系、变量关系和训练效率；
- ▶ 工程上仍需独立处理归一化、窗口、阈值与漂移。

Anomaly Transformer: 异常点与全局序列的关联方式不同



- ▶ Prior association 偏向局部邻近;
- ▶ Series association 从数据学习全局关系;
- ▶ Association discrepancy 区分正常与异常;
- ▶ Minimax 训练放大可分性;
- ▶ 复杂分数仍需业务校准。

深度模型的阈值问题没有消失，只是被推迟了

- ▶ 重构误差、预测误差和关联差异都只是连续分数；
- ▶ 训练损失最低的模型不一定产生最好分离边界；
- ▶ POT/EVT、分位数、验证标签和成本函数会给出不同阈值；
- ▶ 一个全局阈值可能掩盖不同对象/维度的尺度差异；
- ▶ 分数经平滑、聚合和 `point-adjust` 后会显著改变评测；
- ▶ 生产阈值需要结合最低流量、持续性、严重度和告警预算。

反直觉 “更复杂的模型” 经常输给 “更合理的阈值和事件后处理”。

四、研究案例：把检测器带进真实AI系统

真实部署的难点是任务变化、标签稀缺、计算预算和运维偏好，而不仅是模型结构。

Minder: 大规模训练故障发生前，指标模式会先偏离同伴

生产背景

- ▶ 单任务可达上千台机器；
- ▶ 论文环境平均每天约两次故障；
- ▶ 一台机器故障可拖停整个分布式任务；
- ▶ 人工诊断常超过半小时。

Minder 结果

- ▶ 自动发现异常机器；
- ▶ 平均反应时间 3.6 秒；
- ▶ Precision 0.904；
- ▶ F1 0.893；
- ▶ 已在生产环境部署超过一年。

Minder 面对四个困难

1. 故障类型多：GPU、CPU、PCIe、NVLink、NIC、存储、CUDA/NCCL；
2. 正常状态依赖任务：相同温度、利用率在不同训练任务中含义不同；
3. 指标与故障不是一一对应：一个故障影响多个指标，一个指标也对应多个故障；
4. 时序存在噪声：传感器、网络中断、时间戳错位和短暂 jitter 会误导检测。

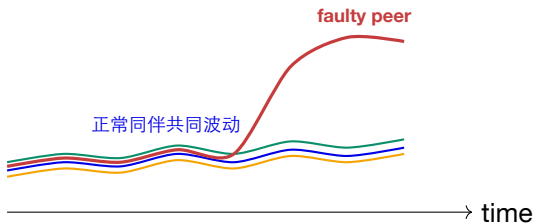
Similarity

Continuity

Per-metric Denoising

Metric Prioritization

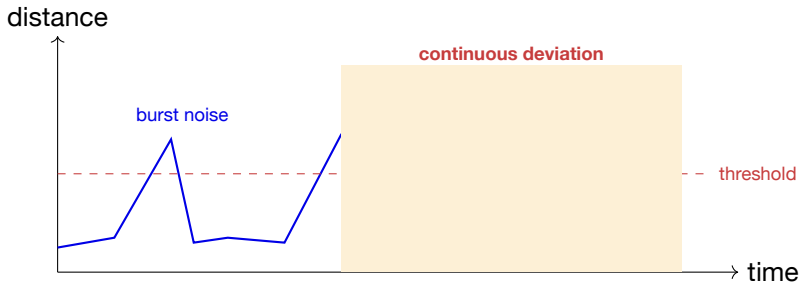
设计 1：同构并行任务中，“同伴”就是动态基线



- ▶ 不需要固定全局阈值；
- ▶ 适应任务负载变化；
- ▶ 找“最不像别人”的机器；
- ▶ 适合数据/张量并行；
- ▶ 共同故障可能一起偏移而漏检。

Peer baseline 是上下文异常检测：数值是否异常取决于同一任务其他机器当时的表现。

设计 2：持续性把故障模式与短暂 jitter 分开



- ▶ 单窗越线只产生候选；
- ▶ 连续多个窗口偏离才判异常机器；
- ▶ 持续性提高 Precision，但增加检测延迟；
- ▶ 不同故障的典型持续时间不同，窗口需校准。

设计 3：每个指标独立去噪，避免“大一统模型”相互干扰

- ▶ CPU、GPU、PFC、吞吐、磁盘、内存的波动形态不同；
- ▶ 不同故障对指标的指示概率不同；
- ▶ Minder 为每个监控指标训练独立 LSTM-VAE；
- ▶ 原始窗口先重构为去噪序列，再做机器间距离；
- ▶ 若把所有指标直接并入一个模型，强噪声维度可能掩盖敏感指标；
- ▶ 多指标结果在“候选层”组合，而不是强迫一个表示解释全部故障。

工程启示 多变量不等于必须单模型；按指标/模态分治再融合，常更易解释和维护。

设计 4：优先检测最敏感指标，降低发现时间

离线

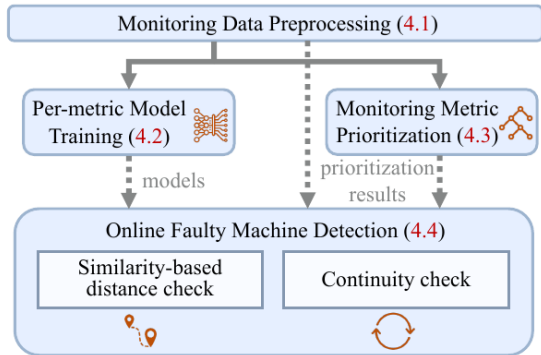
- ▶ 汇总历史故障与指标；
- ▶ 学习哪些指标更常提前异常；
- ▶ 为指标生成优先序；
- ▶ 独立训练对应去噪模型。

在线

- ▶ 先运行高优先级指标；
- ▶ 未发现再逐步扩展；
- ▶ 每个指标做相似性与持续性检查；
- ▶ 找到异常机器后停止搜索。

检测延迟不仅来自窗口长度，也来自要运行多少模型、查询多少指标和等待多少数据。

Minder 系统架构：训练阶段与在线检测阶段分离



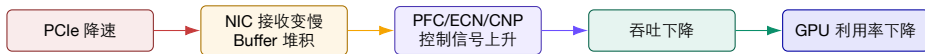
1. 对齐、补齐、归一化;
2. 每指标训练去噪模型;
3. 生成指标优先序;
4. 在线做同伴距离;
5. 检查异常持续性;
6. 输出异常机器。

如何阅读 Minder 的实验结果，而不是只记一个 F1

- ▶ 反应时间：3.6 秒依赖论文采样、窗口、指标优先序和生产环境；
- ▶ **Precision/F1**：真值来自生产故障记录，标签质量会影响结果；
- ▶ 故障类型：硬件故障占论文数据的大部分，迁移到推理集群需复测；
- ▶ 同构假设：同伴应承担相似任务，异构 GPU/并行角色需分组；
- ▶ 共同模式：若所有机器同时受存储或网络上游影响，相对偏离可能不明显；
- ▶ 运营价值：论文目标是异常机器发现，不等于完整根因解释。

研究阅读法 先问“作者解决哪个操作决策”，再读分数；不要把检测器指标直接等同于 MTTR 改善。

Minder 案例：PCIe 降速会沿多指标传播



- ▶ 最显眼的 GPU 利用率下降是传播末端，不是起始根因；
- ▶ 多个指标按时间顺序异常，为后续 RCA 提供传播证据；
- ▶ 第 6 讲只负责发现异常机器/事件，第 7-8 讲继续做根因定位。

ShareAD：深度异常检测在实际使用中遇到的三个问题

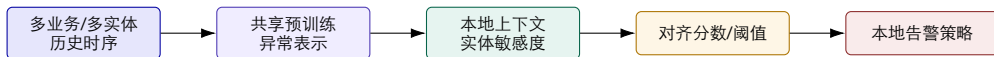
Scalability Gap 成千上万监控实体，逐对象训练模型成本过高。

Availability Gap 新服务、短历史和稀少标签难以独立训练。

Alignment Gap 统一模型分数未必符合本地运维人员的敏感度偏好。

- ▶ ShareAD 提出 Pre-Train-and-Align 范式；
- ▶ 共享预训练模型解决规模和冷启动；
- ▶ 本地上下文与监控实体敏感度用于对齐；
- ▶ 目标是从“每条时序一个模型”转向可共享、可适配的服务。

Pre-Train-and-Align：共享能力与本地偏好分层



- ▶ 表示可以共享，告警代价不能假设完全共享；
- ▶ 新对象可从预训练能力冷启动；
- ▶ 本地反馈应优先调整对齐层，而不是频繁全量重训；
- ▶ 共享模型必须监控域漂移和负迁移。

No Free Lunch: 没有一个检测器对所有时序都最好

- ▶ 数据可能单变量/多变量、周期/非周期、短依赖/长依赖；
- ▶ 异常可能点、区间、上下文、变化点或同伴偏离；
- ▶ 部署可能 CPU 边缘节点，也可能 GPU 后端；
- ▶ 目标可能高 Recall、低 Page 量、低延迟或低资源成本；
- ▶ 超参数对窗口、归一化和数据集高度敏感；
- ▶ 因此“模型选择 + 超参数优化”本身成为工程问题。

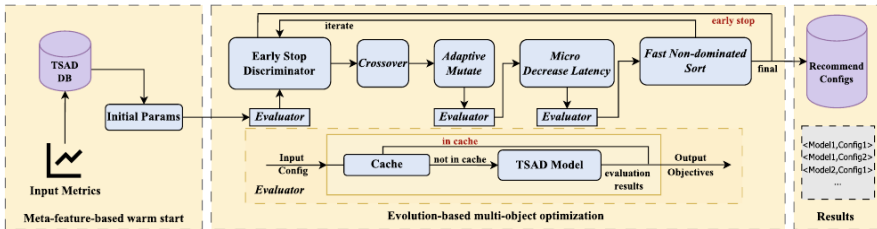
从排行榜转向组合选择 候选模型、阈值、后处理和计算预算应一起进入选择空间。

MOTSAD: 同时优化检测效果与推理延迟

$$\theta^* = \arg \min_{\theta \in \Theta} (-F1(\theta), Latency(\theta))$$

- ▶ 候选算法包括 Isolation Forest、LSTMAD 和 TranAD;
- ▶ 只优化准确率可能选出无法上线的高成本模型;
- ▶ MOTSAD 用多目标进化搜索近似 Pareto 前沿;
- ▶ Meta-feature warm start 从相似历史数据集复用配置;
- ▶ Adaptive mutation、Micro Decrease Latency、Cache 和 Early Stop 提高搜索效率。

MOTSAD 架构：历史配置暖启动 + 面向 TSAD 的多目标搜索



Warm Start Catch22 等元特征检索相似数据集和已有好配置。

Pareto Search 在 F1 与推理时间间保留多种不可支配配置，而不是压成单一权重。

MOTSAD 结果：优化后模型必须同时看 F1 与时间

模型	数据集	F1 前	F1 后	时间前 (ms)	时间后 (ms)
IForest	SMD	0.590	0.808	103.3	95.2
IForest	AI Ops	0.169	0.288	558.7	297.1
LSTMAD	SMD	0.718	0.858	781.8	777.2
LSTMAD	AI Ops	0.678	0.884	3684.4	3563.9
TranAD	SMD	0.672	0.820	11371.2	6227.0
TranAD	AI Ops	0.819	0.826	19890.0	7029.0

- ▶ 不同模型的可优化空间完全不同；
- ▶ TranAD 在示例中主要获得显著延迟下降；
- ▶ 表中数字依赖 SMD/AI Ops、搜索空间和 50 次评估预算；
- ▶ 生产选择应从 Pareto 前沿按 SLO 和成本再决策。

研究案例共同告诉我们：上线检测器要做四层选择

1. 基线选择：历史、自身、同伴、预测或共享预训练；
2. 模型选择：统计、树、VAE、Transformer 或图模型；
3. 决策选择：阈值、持续性、滞回、严重度和告警预算；
4. 运行选择：精度、检测延迟、推理时间、内存、更新成本和解释性。

Minder：同伴与持续性

ShareAD：共享与对齐

MOTSAD：准确率与成本 **Pareto**

五、评测、阈值与上线

异常检测评测最容易被后处理、标签粒度和时间泄漏“制造进步”。

点级混淆矩阵可能与运维体验相反

检测器 A 一次 30 分钟故障只命中 1 个点：点级 Recall 很低，但值班人员收到一次及时通知。

检测器 B 命中故障后 29 分钟：点级 Recall 很高，但几乎没有提前价值。

- ▶ 点级 TP/FP/FN 适合分数比较；
- ▶ 事件级命中更接近“是否发现一次故障”；
- ▶ 事件开始命中时间决定 MTTD；
- ▶ 重复告警数决定值班负担；
- ▶ 必须同时报告点级、区间级和运营指标。

极度不平衡下，Accuracy 几乎没有意义

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}$$

- ▶ 99.9% 时间正常，永远预测正常也有 99.9% Accuracy;
- ▶ 高 Recall 可能通过大量误报获得;
- ▶ 高 Precision 可能只抓最明显异常;
- ▶ PR-AUC 通常比 ROC-AUC 更能体现稀有异常;
- ▶ 不同对象异常率不同，宏平均与微平均含义不同;
- ▶ F1 未表达检测延迟、严重度和告警重复。

Range-based Precision/Recall: 区间重叠程度也应进入评分

- ▶ 一个异常区间被命中至少一次，体现 existence reward；
- ▶ 命中区间多少比例，体现 overlap reward；
- ▶ 越早命中可赋予更高 positional reward；
- ▶ 把一个事件切成多个预测片段，应受到 cardinality penalty；
- ▶ 参数选择必须公开，否则不同论文的“Range-F1”不可直接比较。

教学建议 报告 Event Recall、平均首次检测延迟、每事件重复通知数，再补充 point/range F1。

检测延迟：命中得太晚，仍然可能没有运维价值

$$Delay_i = t_i^{first\ detection} - t_i^{event\ start}$$

至少报告

- ▶ Median / P95 delay;
- ▶ 漏检事件比例;
- ▶ 故障前预警提前量;
- ▶ 数据采集 + 特征 + 推理总延迟。

不要混淆

- ▶ 模型推理时间;
- ▶ 观察窗口等待时间;
- ▶ 告警 for 持续时间;
- ▶ 通知与人工确认时间。

Point-adjustment 可能制造“看起来很高”的 F1

- ▶ 常见做法：只要命中异常区间一个点，就把整个区间都算命中；
- ▶ 这会让晚到、稀疏或随机命中的模型获得高 Recall；
- ▶ 不同论文的后处理不一致，模型排名可能因此改变；
- ▶ 阈值在测试集上优化会进一步放大虚高；
- ▶ 数据集中的重复模式、人工异常和标签错误也会产生错觉。

复查要求 保留原始分数、原始点预测、后处理预测和事件映射，确保每一步可重放。

阈值选择必须在验证集完成，并进行敏感性分析

1. 在训练基线上拟合模型；
2. 在独立验证窗上确定阈值、平滑和持续性；
3. 绘制阈值–Precision–Recall–告警量曲线；
4. 选择满足最低 Recall 与最大 Page 预算的区域；
5. 在未来测试窗冻结配置；
6. 报告阈值小幅变化时结论是否稳定。

如果阈值改变 2% 就让 F1 或告警量翻倍，该检测方案高度脆弱，不适合直接上线。

漂移监控：检测器本身也需要可观测性

输入漂移

- ▶ 指标分布、缺失率、采样率；
- ▶ 新对象/硬件/租户；
- ▶ 季节性和业务日历；
- ▶ 特征相关结构。

输出漂移

- ▶ 分数分布与越线率；
- ▶ 每对象告警频率；
- ▶ 专家确认/拒绝比例；
- ▶ 未命中事件与延迟；
- ▶ 模型版本和阈值版本。

记录误报与漏报 误报/漏报必须回填到“对象 + 时间窗 + 模型版本 + 阈值版本”，否则无法真正学习。

上线策略：Shadow 先看行为，Canary 再改变通知

1. **Offline replay**: 历史事件和正常高峰回放；
2. **Shadow**: 实时打分但不发通知，比较旧规则与新模型；
3. **Silent evaluation**: 让值班人员标注“若收到是否有用”；
4. **Canary**: 仅少量对象/团队接收新告警；
5. **Dual run**: 保留旧规则作为安全网；
6. **Rollback**: 模型、阈值和后处理均可独立回滚。

异常检测上线的风险不是模型崩溃，而是它悄悄改变团队的注意力分配。

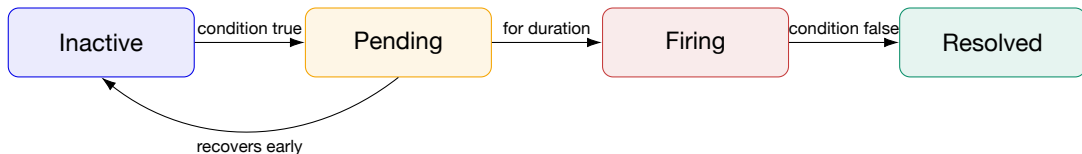
可复现实验包：让结果能被下一位同学重放

制品	最低要求
数据 切分 特征 模型 决策 评测 运行	原始时间范围、对象、单位、采样、缺失、标签来源与脱敏说明 Train/Validation/Test 的明确时间边界与外推场景 窗口、归一化参数、周期/外生变量和版本 代码、随机种子、超参数、依赖与硬件 阈值、平滑、持续性、滞回、事件合并规则 point/range/event 指标、延迟、告警量和置信区间 推理时间、CPU/GPU/内存、失败模式和回滚方法

六、从异常分数到告警事件

告警治理的目标不是“压得越少越好”，而是在保留关键事件的同时减少无效告警处理量。

检测器输出需要经过告警状态机



- ▶ 分数越线不应立即等于通知；
- ▶ Pending 吸收短暂抖动；
- ▶ Resolved 也应保留恢复时间和恢复证据；
- ▶ 状态转换本身要带模型/规则版本。

for 与 滞回：减少阈值附近反复 Firing/Resolved

持续性 **for** 条件连续满足 5 分钟才 Firing；适合过滤短峰，但增加 MTTR。

滞回 **Hysteresis** 超过 0.8 触发，降到 0.6 以下才恢复；避免在单一阈值附近抖动。

- ▶ 触发阈值与恢复阈值应分开；
- ▶ 数据缺失时是保持 Pending、恢复还是进入 Unknown，需要明确定义；
- ▶ 短暂恢复是否重置 for，会显著影响告警量；
- ▶ 严重故障可用高阈值快速通道，不必统一等待。

告警身份决定能否去重：哪些标签属于“同一件事”？

适合参与身份

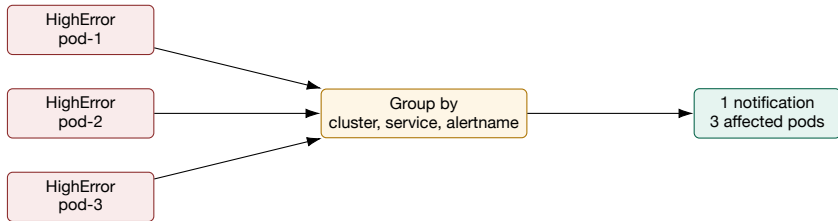
- ▶ alertname / service / cluster;
- ▶ namespace / component;
- ▶ severity / tenant（视路由需求）;
- ▶ 稳定的资源标识。

不适合参与身份

- ▶ request_id / trace_id;
- ▶ 当前值、时间戳;
- ▶ Pod UID 是否入键，取决于实例级还是服务级告警;
- ▶ 高基数动态描述文本。

失败模式 身份过细无法去重；身份过粗会把不同租户、区域或故障压成一个事件。

Grouping: 把同类告警合成一次通知



- ▶ `group_wait` 等待相关告警到齐;
- ▶ `group_interval` 控制组内新变化的通知频率;
- ▶ `repeat_interval` 控制未恢复事件重复提醒;
- ▶ 分组键应表达一个值班人员能共同处理的故障范围。

Inhibition 与 Silence：一个按规则抑制通知，一个按时间静默

Inhibition 当 ServiceDown 已触发，抑制同服务的 HighLatency/HighError 通知；底层告警仍存在。

- ▶ 依赖 source/target matchers；
- ▶ 要求对象标签一致；
- ▶ 适合已确认的依赖症状；匹配本身不证明因果。

Silence 在维护窗口按 matcher 暂停通知。

- ▶ 必须有到期时间与原因；
- ▶ 不应删除检测数据；
- ▶ 维护结束后检查残留静默。

Alertmanager 当前配置推荐使用 `matchers/source_matchers/target_matchers`；旧 `match` 字段已标记 Deprecated。

跨规则关联：时间相近不等于同一个事件

- ▶ 时间：在同一故障活动窗口内；
- ▶ 对象：同一服务、节点、租户、区域或调用链；
- ▶ 拓扑：存在调用、部署、宿主或网络邻接；
- ▶ 变更：共享一次发布、配置、扩缩容或维护；
- ▶ 语义：规则含义、症状模式和处置团队相近；
- ▶ 因果方向：上游错误先于下游超时，不能仅按共现无向聚类。

最低事件模型 event_id、对象集合、开始/结束、首发告警、关联告警、变更、SLO 影响和证据引用。

告警聚类方法：规则、图、统计与语义各有边界

方法	优点	风险
固定规则	可解释、低延迟、易审计	属性组合爆炸，长期维护困难
时间窗口	简单、适合风暴初筛	无关并发事件被错误合并
拓扑图	保留系统关系与传播路径	拓扑过期、异步链路和共享依赖复杂
共现图/图学习	适应多属性组合与历史模式	低频关键告警、漂移与黑盒分数
文本/Embedding	能聚接口约定义规则和描述	文本模板质量、版本和语义幻觉
混合方法	可同时利用图、时间和知识	需要清晰的在线/离线成本分工

AlertGuardian: 治理告警完整生命周期



- ▶ 在线图模型：过滤高频、周期和重复噪声；
- ▶ RAG + LLM：把剩余关键告警变成可行动摘要；
- ▶ 离线多智能体：去重、聚合、调阈值、加时间条件；
- ▶ 复核智能体与 SRE 审批降低误压制风险；仍须抽查漏报并保留告警恢复入口。

研究证据：告警风暴既有持续噪声，也有周期噪声

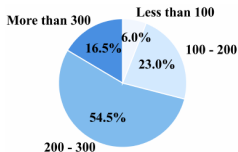


Fig. 3: The percentage breakdown of average per-minute alert volume in *Company-X*.

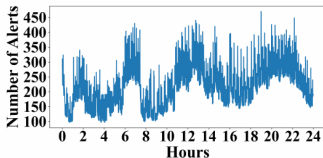


Fig. 4: Change trend of alert volume under 59,607 alert rules in system A.

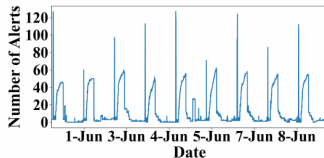


Fig. 5: Change trend of alert volume under 3,544 alert rules in system B.

- ▶ 论文调研 *Company-X* 超过 200 个系统，9 天数据超过 20GB；
- ▶ 超过 70% 的系统平均每分钟至少 200 条告警；
- ▶ *System A* 存在持续高告警量；*System B* 出现日周期峰值；
- ▶ 静态阈值和冗余规则让“正常模式”不断触发通知。

AlertGuardian 总体架构：小模型处理高吞吐，LLM 提供上下文深度

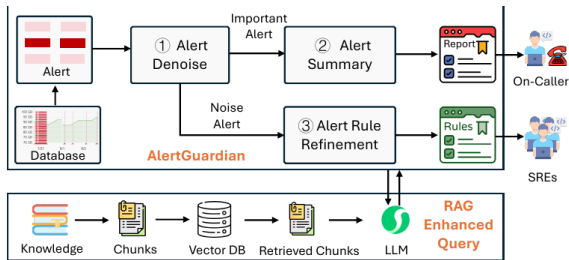


Fig. 9: Overview of AlertGuardian in Company-X.

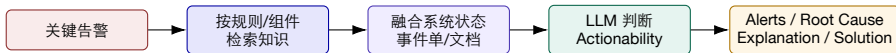
- ▶ Alert Denoise: 在线;
- ▶ Alert Summary: 关键告警;
- ▶ Rule Refinement: 离线;
- ▶ RAG 接入内部知识;
- ▶ SRE 保留审批权;
- ▶ 不让 LLM 承担全部流量。

Alert Denoise: 虚拟噪声节点让“常见但无故障”成为可学习模式

1. 将告警按 1 分钟窗口离散化；
2. 对持续未恢复告警复制到后续窗口，保留持续性；
3. 每分钟注入虚拟噪声告警，代表稳定持久噪声；
4. 告警属性值作为图节点，共现形成边；
5. LINE + Transformer 学习局部和高阶关系；
6. 与虚拟噪声节点相似度高的告警被判断为噪声候选。

关键权衡 θ 越高过滤越多但可能漏关键告警；论文默认 0.7，是特定数据的工程选择。

Alert Summary: 降噪后仍要回答“发生了什么、为什么、怎么办”



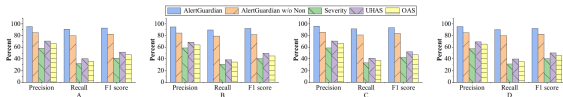
- ▶ 无需处置的告警可以被摘要为“无需动作”；
- ▶ 需要处置时，输出根因候选、解释、方案和知识链接；
- ▶ RAG 降低专有知识缺失，但仍需防止错误引用和过度确定。

Alert Rule Refinement: 多智能体优化规则，但必须回放验证

1. Detect Agent: 聚类噪声模式与周期模式;
2. RAG Agent: 检索规则解释、事故单和系统知识;
3. Rule Agent: 提出规则去重、聚合、阈值和时间条件修改;
4. Review Agent: 语法检查 + 历史告警仿真;
5. 若压制关键告警或降噪不足，反馈重新迭代;
6. 最终建议仍由 SRE 审批，避免自动修改生产规则。

四类策略 Rule Deduplication、Rule Aggregation、Threshold Adjustment、Temporal Analysis。

AlertGuardian 结果：降噪数量必须与关键告警保留一起读



- ▶ 四个系统降噪率 93.82–95.50%；
- ▶ 摘要 Action Accuracy 98.5%；
- ▶ RCA Accuracy 90.5%；
- ▶ 优化 1174 条规则；
- ▶ 375 条被 SRE 接受，采纳率 32%；
- ▶ 去重建议采纳率最高。

AlertGuardian 的迁移边界：95% 降噪不是通用目标

- ▶ 数据来自单一公司四类业务，外部系统需重新标注和校准；
- ▶ 关键告警真值依赖事故报告与 SRE 标注，存在主观性；
- ▶ 在线图模型与属性匿名化对高基数场景重要，但会损失部分语义；
- ▶ LLM 摘要和规则建议依赖知识库质量与企业权限；
- ▶ 规则阈值修改可能降低 Recall，必须历史回放与人工审批；
- ▶ 降噪率应与 Incident Recall、MTTD、重复 Page 和误抑制共同评估。

最危险的 KPI 若团队只追求“压缩率”，最简单的办法是关闭告警；这显然不等于可靠性提升。

从异常检测进入后续 RCA：交付的是事件上下文，不是一串分数

字段	内容
Identity	event_id、cluster/service/component、tenant、模型/规则版本
Time	基线窗、首次异常、Firing、恢复、相关变更时间
Detection	原始分数、阈值、持续性、触发指标与同伴比较
Scope	影响对象、SLO、用户/任务、严重度和置信度
Correlation	归并告警、拓扑邻接、Trace、日志、发布与配置事件
Counter-evidence	正常信号、未满足条件、被排除候选
Action	Runbook、下一步调查、抑制/静默原因和所有者
Feedback	SRE 确认、误报/漏报、最终根因和恢复验证

第 7 讲将使用这些字段生成根因候选、传播路径和诊断证据。

课堂练习：把 1000 条告警压成一个可行动事件

输入

- ▶ 500 个 Pod HighLatency;
- ▶ 300 个 DownstreamTimeout;
- ▶ 180 个 RetryRateHigh;
- ▶ 20 个 RedisConnectionPoolFull;
- ▶ 5 分钟前发生 Redis 配置变更。

要求

1. 定义分组键;
2. 选择首发/主告警;
3. 写两条抑制关系;
4. 生成事件摘要;
5. 给出一个反证问题;
6. 定义恢复验收指标。

参考方向 不要按 Pod 建 1000 个 Incident; 以业务服务 + Redis 依赖 + 变更窗口组织证据。

最小实现骨架：稳健分数 + 持续性 + 事件合并

```
baseline = rolling_median(x, window=7*24*60)
scale     = rolling_mad(x, window=7*24*60) + eps
score     = 0.6745 * abs(x - baseline) / scale

candidate = score > 4.0
firing     = consecutive(candidate, minutes=5)
resolved   = consecutive(score < 2.5, minutes=10)

event_key = (cluster, service, alert_family)
event     = merge_alerts(event_key, firing, within="10m")
```

- ▶ 代码只是采集字段约定示意；
- ▶ 生产还需缺失值、冷启动、季节性、版本和低流量处理；
- ▶ 任何阈值都必须通过验证集和告警预算校准。

事件评测算例：告警压得更少，为什么召回反而下降？

测试集有 5 起真实故障；用预先固定的时间容差做一对一告警匹配，重复通知另计负担。

策略	发出告警	匹配真实故障	漏检 / 误报
A	10	4	漏检 1；误报 6
B	3	3	漏检 2；误报 0
A 指标	Precision=4/10	Recall=4/5	F1=0.533
B 指标	Precision=3/3	Recall=3/5	F1=0.750

B 的通知数减少 70%，但多漏掉了一起真实故障。

推导与判断 即使 B 的 F1 更高，也不能直接上线：还应比较漏掉的是哪类故障、业务影响及检测延迟，按故障等级确定接受条件。

算例使用假设数据。

参考资料与延伸阅读

- ▶ Liu, Ting, Zhou, *Isolation Forest*, ICDM 2008; scikit-learn stable IsolationForest API;
- ▶ Taylor and Letham, *Forecasting at Scale*; Prophet 官方 Quick Start / Outliers 文档;
- ▶ Tatbul et al., *Precision and Recall for Time Series*, NeurIPS 2018;
- ▶ Wu and Keogh, *Current Time Series Anomaly Detection Benchmarks are Flawed*;
- ▶ Tuli et al., *TranAD*, PVLDB 2022; Xu et al., *Anomaly Transformer*, ICLR 2022;
- ▶ Deng et al., *Minder*, NSDI 2025;
- ▶ Huang, Chen, Li, *SLA-VAE*, WWW 2022; He, Chen, Zheng, *ShareAD*, TOSEM 2025;
- ▶ Tang, Tan, Chen, *MOTSAD*, ICSOC 2025;
- ▶ Yu et al., *AlertGuardian*, ASE 2025 / arXiv 2026;
- ▶ Prometheus Alertmanager 官方 Overview/Configuration; 本地 ai-infra-engineer-learning-main Alerting materials。

官方链接: [scikit-learn IsolationForest](#); [Prophet Docs](#); [Alertmanager](#); [AlertGuardian](#)

好的异常检测不
是发出更多告警
而是更早交付更
少、更可信的事件

校企联合研究生课程 面向 AI 原生系统的智能运维